DOCUMENT CLASSIFICATION USING CONVOLUTIONAL NEURAL NETWORK

Contents

1. Introduction 1

2. Data description. 2

3. Data preprocessing and Augmentation 2

3.1 Data preprocessing 2

3.2 Augmentation 2

4. Model Architecture 3

4.1 Base Model and Transfer Learning 3

4.2 Custom Layers and model design 3

5. Training and Evaluation 4

5.1 Training setup 4

5.2 Evaluation Metrics 4

6. Results 4

7. Model Improvement and Optimization 5

7.1 Fine-tuning attempt 5

8. Challenges Faced 5

9. Conclusion 6

10. References 6

11. GitHub 7

1. Introduction

Document classification is vital to automating business operations such as sorting emails, invoices and reports. Some modern-day institutions utilize document classification to streamline workflows mostly common in the finance, health and insurance enterprises. Manual classification is time consuming. Deep learning particularly convolutional neural networks (CNN) has demonstrated robust performance when it comes to image classification (LeCun, et al., 2015).

2. Data description.

The dataset used in this study is the RVL-CDIP dataset, accessed through Hugging Face. It consists of scanned document images across 16 document categories such as invoices, emails, and forms. This dataset reflects real-world document processing challenges. (Harley, et al., 2015)

3. Data preprocessing and Augmentation

3.1 Data preprocessing

Once the dataset was loaded into the Jupyter Notebook using the Hugging Face Datasets library, the images and their corresponding labels were stored locally to ensure efficient access during training. Storing the data locally allows for faster input/output operations when providing images to the model.

Images were resized to 224 × 224 pixels, ensuring compatibility with pretrained convolutional neural networks such as MobileNetV2. Standardizing image dimensions is essential, as neural networks require fixed input sizes. Additionally, resizing helps maintain consistency across the dataset.

Pixel values were normalized using a rescaling factor of 1./255 across all data generators (training, validation, and testing). This converts pixel values from the range [0, 255] to [0, 1], which improves numerical stability and allows the neural network to converge more efficiently during training. Smaller input values help prevent large gradient updates and improve overall learning performance.

3.2 Augmentation

To improve the model's generalization strength and decrease overfitting, limited data augmentation approaches were used on the training dataset. Specifically, a rotation range of ±3 degrees and a zoom range of ±5% (0.05) were used. These transformations introduce slight variations in the training images, enabling the model to become more robust to minor distortions such as scanning inconsistencies or slight misalignments (Shorten & Khoshgoftaar, 2019).

The rotation augmentation helps the model handle small angular deviations commonly found in scanned documents, while the zoom augmentation improves robustness to variations in scale. However, heavy augmentation techniques were intentionally avoided, as document-based datasets are sensitive to structural changes, and excessive transformations could distort important features such as layout and text positioning.

Augmentation was not utilized on the validation or test datasets. The validation dataset must remain clean and unchanged for a fair and impartial evaluation of the model during training. Similarly, the test dataset represents completely unseen real-world data, and therefore must not be altered, ensuring that performance metrics accurately reflect the model's true predictive capability.

4. Model Architecture

4.1 Base Model and Transfer Learning

Images were loaded using data generators via flow_from_directory, where the target size was set to 224 × 224, the batch size to 32, and the class mode to categorical. The batch size determines how many images are processed at once, balancing computational efficiency and memory usage. The categorical class mode converts class labels into one-hot encoded vectors, which are required for multi-class classification tasks.

For the test dataset, shuffling was disabled to preserve the original order of the images. This is imprtant when generating predictions, as it ensures that outputs can be correctly matched with their corresponding filenames and labels.

The pretrained MobileNetV2 model was selected as the base model due to its efficiency and suitability for resource-constrained environments (Sandler, et al., 2018). The model was equated with ImageNet weights from ImageNet, allowing it to leverage previously learned features such as edges, textures, and shapes. This use of transfer learning reduces training time and improves performance, especially when working with limited data (Jialin & Yang, 2010).

The parameter include_top=False was utilized to eliminate the initial classification layer of the pretrained model, enabling the addition of a custom classifier tailored to the specialized task. The input shape was defined as (224, 224, 3) to match the resized RGB images. Furthermore, the foundation model was frozen (trainable = False) to protect the pretrained weights and conserve them from being changed throughout training, ensuring the mastered low-level features remain intact.

4.2 Custom Layers and model design

The output feature maps from the base model were first passed through a Global Average Pooling layer, which converts them into a one feature vector. This minimizes the number of parameters and helps mitigate overfitting while retaining essential spatial information.

A couple of fully connected (Dense) layers and Dropout layers were then added to form a robust classification head. Dropout is a regularization technique that randomly disables a proportion of neurons during training, helping to decrease overfitting by encouraging the model to master more generalized features (Srivastava, et al., 2014).

The architecture consists of:

- A first dense layer containing 512 neurons and ReLU activation, designed to learn hard patterns in the dataset.
- A Dropout layer (50%), which disables half of the neurons to improve generalization.
- A second dense layer comprising of 256 neurons and relu activation, paired with a Dropout layer (40%) to further reduce overfitting and compress learned features.
- A third dense layer having 128 neurons and relu activation, enabling more refined characteristics mining, accompanied by a Dropout layer (30%).

The last output layer comprises of 16 neurons, matching the number of categories in the data, with a softmax activation function. This converts the model's product into a probability distribution across all categories, letting it to forecast the class for each image.

Overall, the model combines a pretrained feature extractor with a custom classification head, leveraging the strengths of transfer learning while adapting to the specific dataset and classification task.

5. Training and Evaluation

5.1 Training setup

The model was compiled using the Keras framework with a learning rate of 0.0001 to ensure stable and gradual learning, which is particularly important when using transfer learning. The categorical cross entropy loss function was selected for multi-class classification with one-hot encoded labels, while accuracy was used as the evaluation metric to measure prediction correctness.The Adam optimizer was chosen due to its efficiency, combining the strengths of adaptive optimization methods such as AdaGrad and RMSProp (Kingma & Ba, 2014).

To improve training performance, two callbacks were implemented. ModelCheckpoint monitored val_accuracy and saved only the best-performing model weights (save_best_only=True, mode="max"), reducing storage while preserving optimal performance. EarlyStopping monitored val_loss with a patience of 3 epochs and restored the best weights, helping to prevent overfitting and reduce unnecessary training time.

The model was trained using model.fit(), where the training dataset updated model weights and the validation dataset evaluated performance after each epoch. Training was set for a maximum of 30 epochs, with callbacks controlling early termination if performance stopped improving.

5.2 Evaluation Metrics

Model performance was evaluated mainly using accuracy and loss. Accuracy measures the proportion of correctly classified images, providing a clear indication of predictive performance. Loss, on the other hand, reflects the error between predicted and actual outputs, offering insight into how well the model is learning during training.

6. Results

The initial model achieved an accuracy of approximately 51%, which was later improved to around 60.8%. The loss went from 1.38 to 1.3. These improvements were primarily due to adjustments in preprocessing steps. The model performed well on visually distinct documents but struggling with similar, text-heavy classes, showing limitations of CNNs in capturing semantic and structural differences. High-performing classes such as email (F1-score: 0.84), file folder (0.82), and advertisement (0.79) were classified accurately due to their distinct visual structures.

Precision measures accuracy, recall measures well the model finds examples, F1-score summarizes the precision and recall.

```
100/100 [==============================] - 24s 234ms/step
                        precision    recall  f1-score   support

         advertisement       0.85      0.73      0.79       200
                budget       0.58      0.48      0.53       200
                 email       0.84      0.83      0.84       200
           file folder       0.76      0.90      0.82       200
                  form       0.46      0.41      0.43       200
           handwritten       0.55      0.90      0.68       200
               invoice       0.61      0.46      0.52       200
                letter       0.44      0.54      0.48       200
                  memo       0.31      0.42      0.36       200
          news article       0.77      0.69      0.73       200
          presentation       0.56      0.58      0.57       200
         questionnaire       0.50      0.40      0.44       200
                resume       0.73      0.65      0.69       200
scientific publication       0.65      0.83      0.73       200
     scientific report       0.56      0.22      0.32       200
         specification       0.70      0.68      0.69       200

              accuracy                           0.61      3200
             macro avg       0.62      0.61      0.60      3200
          weighted avg       0.62      0.61      0.60      3200
```

Figure 1(classification report)

7. Model Improvement and Optimization

Several modifications were made to improve model performance. The image size was increased from 128 × 128 to 224 × 224, allowing the model to capture more detailed visual characteristics. The learning rate was reduced from 0.0003 to 0.0001, resulting in more stable training. Additionally, the batch size was decreased from 128 to 32, improving generalization and reducing memory constraints. The model architecture was also refined to enhance feature learning. These changes had a significant impact on performance, as hyperparameter tuning plays a crucial role in optimizing deep learning models (Bergstra & Bengio, 2012)

7.1 Fine-tuning attempt

Attempted to unfreeze layers, but performance decreased, as fine-tuning can cause catastrophic forgetting if not applied properly.

8. Challenges Faced

Several challenges were encountered during this project. Training time was relatively long, taking approximately 148 minutes, which reflects the computational demands of deep learning models. The model also initially produced low accuracy, requiring multiple adjustments to improve performance.

Handling the dataset presented additional time complexity, as images needed a lot of time to be stored locally to ensure accessibility during training. Furthermore, document classification is inherently more challenging than standard object classification tasks, as it involves not only visual features but also structural elements such as layout and text positioning.

9. Conclusion

The model was successfully developed and achieved a moderate performance of approximately 60% accuracy, demonstrating the effectiveness of the overall approach. The results highlight the importance of proper data preprocessing and hyperparameter tuning in improving model performance. However, convolutional neural networks such as MobileNetV2 can struggle with text-heavy document images, as they are primarily designed for visual feature extraction rather than textual understanding.

This suggests that document classification tasks may benefit from more advanced or multimodal approaches that combine both visual and textual information (Xu, et al., 2020). In future work, more powerful architectures such as EfficientNet (Tan & Le, 2020) could be explored to improve performance. Additionally, integrating Optical Character Recognition (OCR) with text-based or transformer-based models may further enhance the model's ability to interpret document content.

Overall, the model demonstrates practical value and could be applied in real-world domains such as insurance, healthcare, and financial services, where automated document classification can improve efficiency and reduce manual processing.

10. References

Bergstra, J. & Bengio, Y., 2012. Random Search for Hyper-Parameter Optimization. Journal of Machine Learning Research, 13(10), p. 25.

Harley, A. W., Ufkes, A. & Derpanis, K. G., 2015. Evaluation of Deep Convolutional Nets for Document Image Classification and Retrieval. 25 Febraury .p. 9.

Jialin, S. & Yang, Q., 2010. A Survey on Transfer Learning. IEEE TRANSACTIONS ON KNOWLEDGE AND DATA ENGINEERING, October , 22(10), p. 15.

Keras, n.d. Keras Documentation: Keras 3 API Documentation. [Online]
Available at: https://keras.io/api
[Accessed 28 March 2026].

Kingma, D. P. & Ba, J. L., 2014. A Method for Stochastic Optimization. 22 December.p. 15.

LeCun, Y., Bengio, Y. & Hinton, G., 2015. Deep Learning. May.521(7553).

Sandler, M. et al., 2018. MobileNetV2: Inverted Residuals and Linear Bottlenecks. 13 January.p. 14.

Shorten, C. & Khoshgoftaar, T. M., 2019. A survey on Image Data Augmentation for Deep Learning. Journal of Big Data, 1 July, 6(1), p. 48.

Srivastava, N. et al., 2014. Dropout: A Simple Way to Prevent Neural Networks from Overfitting. Journal of Machine Learning Research, 15(56), p. 30.

Tan, M. & Le, Q. V., 2020. EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. 11 September.p. 11.

Tensorflow, n.d. Docs4dev. [Online]
Available at:
https://www.docs4dev.com/docs/tensorflow/2.3.0/keras/preprocessing/image/imagedatagenerator.html
[Accessed 28 March 2026].

Xu, Y. et al., 2020. LayoutLM: Pre-Training of Text and Layout for Document Image Understanding. 16 June.p. 9.

11. GitHub

GITHUB link: https://github.com/Carlson29/Document-Classification-Model.git